

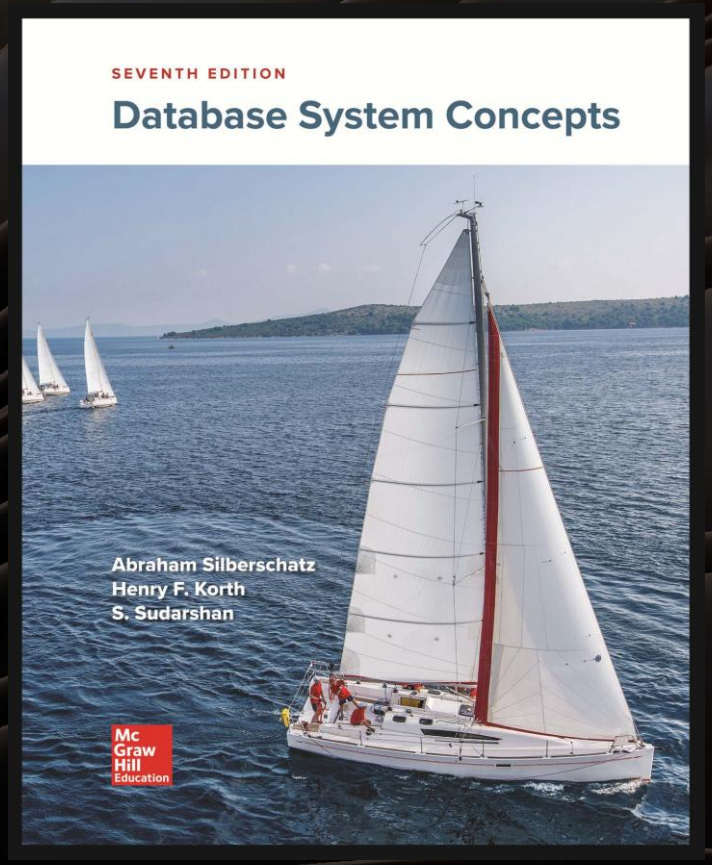
# IF2040 – Pemodelan Basis Data

# SQL (Part 3)

©2026 - Tim Pengajar IF2040 Semester I TA 2026/2027

©2026 - Tim Pengajar IF2040 Pemodelan Basis Data

KNOWLEDGE & SOFTWARE ENGINEERING



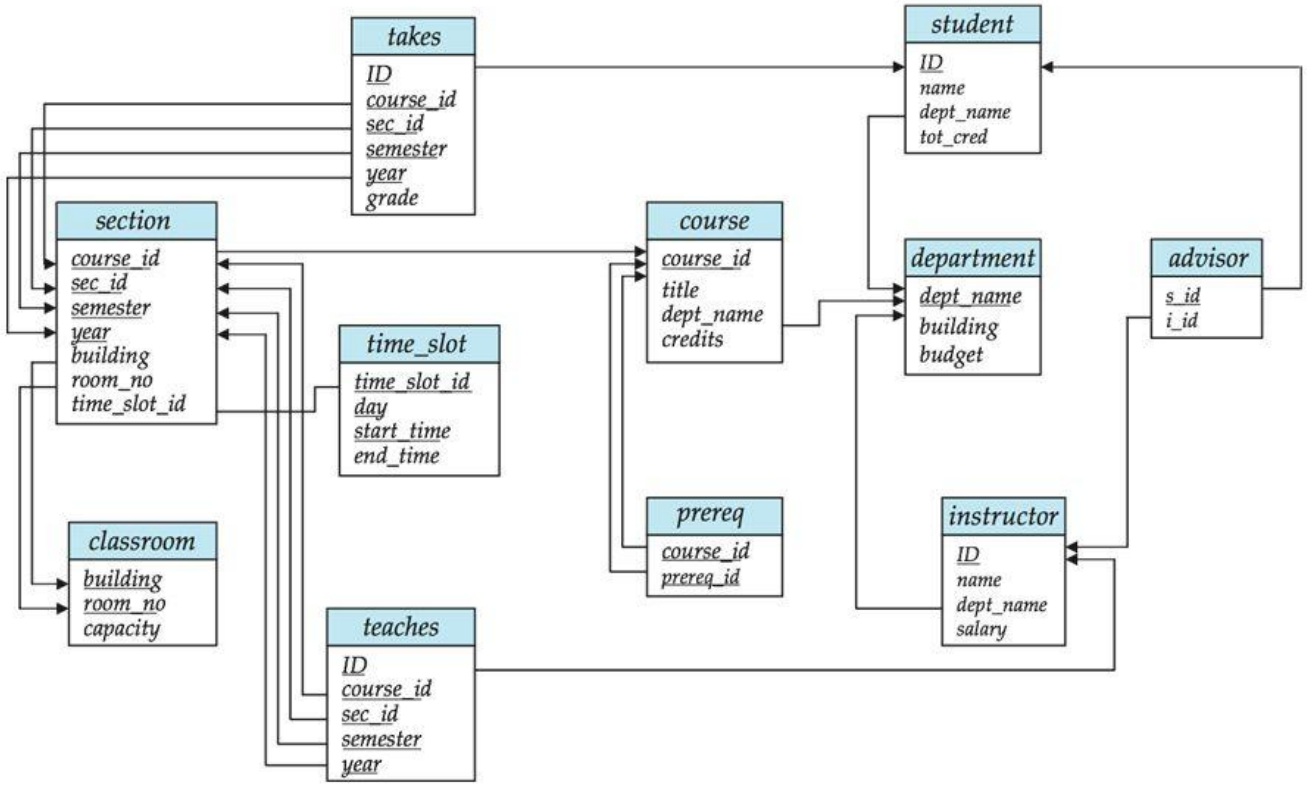

# Sumber

Silberschatz, Korth, Sudarshan: "Database System Concepts", 7th Edition

- Chapter 3 : Introduction to SQL
- Chapter 4 : Intermediate SQL

©2026 - Tim Pengajar IF2040 Pemodelan Basis Data

# Schema Diagram






©2026 - TIM PENGAJAR IF2040 PEMODELAN BASIS DATA

# Joined Relations

**Join operations** take two relations and return as a result another relation.

A join operation is a Cartesian product which requires that tuples in the two relations match (under some condition). It also specifies the attributes that are present in the result of the join

The join operations are typically used as subquery expressions in the **from** clause

Three types of joins:

- Natural join
- Inner join
- Outer join



KNOWLEDGE & SOFTWARE ENGINEERING

# Natural Join in SQL

Natural join matches tuples with the same values for all common attributes, and retains only one copy of each common column.

List the names of instructors along with the course ID of the courses that they taught

- **select** *name, course_id*
  **from** *students, takes*
  **where** *student.ID = takes.ID*;

Same query in SQL with "natural join" construct

- **select** *name, course_id*
  **from** *student* **natural join** *takes*;



KNOWLEDGE & SOFTWARE ENGINEERING

# Natural Join in SQL (Cont.)

The **from** clause can have multiple relations combined using natural join:

**select** $A_1, A_2, \ldots A_n$
**from** $r_1$ **natural join** $r_2$ **natural join** .. **natural join** $r_n$
**where** $P$ ;



KNOWLEDGE & SOFTWARE ENGINEERING

# Student Relation

| *ID* | *name* | *dept_name* | *tot_cred* |
|---|---|---|---|
| 00128 | Zhang | Comp. Sci. | 102 |
| 12345 | Shankar | Comp. Sci. | 32 |
| 19991 | Brandt | History | 80 |
| 23121 | Chavez | Finance | 110 |
| 44553 | Peltier | Physics | 56 |
| 45678 | Levy | Physics | 46 |
| 54321 | Williams | Comp. Sci. | 54 |
| 55739 | Sanchez | Music | 38 |
| 70557 | Snow | Physics | 0 |
| 76543 | Brown | Comp. Sci. | 58 |
| 76653 | Aoi | Elec. Eng. | 60 |
| 98765 | Bourikas | Elec. Eng. | 98 |
| 98988 | Tanaka | Biology | 120 |

©2026 - Tim Pengajar IF2040 Pemodelan Basis Data

# Takes Relation

| *ID* | *course_id* | *sec_id* | *semester* | *year* | *grade* |
|---|---|---|---|---|---|
| 00128 | CS-101 | 1 | Fall | 2017 | A |
| 00128 | CS-347 | 1 | Fall | 2017 | A- |
| 12345 | CS-101 | 1 | Fall | 2017 | C |
| 12345 | CS-190 | 2 | Spring | 2017 | A |
| 12345 | CS-315 | 1 | Spring | 2018 | A |
| 12345 | CS-347 | 1 | Fall | 2017 | A |
| 19991 | HIS-351 | 1 | Spring | 2018 | B |
| 23121 | FIN-201 | 1 | Spring | 2018 | C+ |
| 44553 | PHY-101 | 1 | Fall | 2017 | B- |
| 45678 | CS-101 | 1 | Fall | 2017 | F |
| 45678 | CS-101 | 1 | Spring | 2018 | B+ |
| 45678 | CS-319 | 1 | Spring | 2018 | B |
| 54321 | CS-101 | 1 | Fall | 2017 | A- |
| 54321 | CS-190 | 2 | Spring | 2017 | B+ |
| 55739 | MU-199 | 1 | Spring | 2018 | A- |
| 76543 | CS-101 | 1 | Fall | 2017 | A |
| 76543 | CS-319 | 2 | Spring | 2018 | A |
| 76653 | EE-181 | 1 | Spring | 2017 | C |
| 98765 | CS-101 | 1 | Fall | 2017 | C- |
| 98765 | CS-315 | 1 | Spring | 2018 | B |
| 98988 | BIO-101 | 1 | Summer | 2017 | A |
| 98988 | BIO-301 | 1 | Summer | 2018 | *null* |

©2026 - Tim Pengajar IF2040 Pemodelan Basis Data

# *student* natural join *takes*

| *ID* | *name* | *dept_name* | *tot_cred* | *course_id* | *sec_id* | *semester* | *year* | *grade* |
|---|---|---|---|---|---|---|---|---|
| 00128 | Zhang | Comp. Sci. | 102 | CS-101 | 1 | Fall | 2017 | A |
| 00128 | Zhang | Comp. Sci. | 102 | CS-347 | 1 | Fall | 2017 | A- |
| 12345 | Shankar | Comp. Sci. | 32 | CS-101 | 1 | Fall | 2017 | C |
| 12345 | Shankar | Comp. Sci. | 32 | CS-190 | 2 | Spring | 2017 | A |
| 12345 | Shankar | Comp. Sci. | 32 | CS-315 | 1 | Spring | 2018 | A |
| 12345 | Shankar | Comp. Sci. | 32 | CS-347 | 1 | Fall | 2017 | A |
| 19991 | Brandt | History | 80 | HIS-351 | 1 | Spring | 2018 | B |
| 23121 | Chavez | Finance | 110 | FIN-201 | 1 | Spring | 2018 | C+ |
| 44553 | Peltier | Physics | 56 | PHY-101 | 1 | Fall | 2017 | B- |
| 45678 | Levy | Physics | 46 | CS-101 | 1 | Fall | 2017 | F |
| 45678 | Levy | Physics | 46 | CS-101 | 1 | Spring | 2018 | B+ |
| 45678 | Levy | Physics | 46 | CS-319 | 1 | Spring | 2018 | B |
| 54321 | Williams | Comp. Sci. | 54 | CS-101 | 1 | Fall | 2017 | A- |
| 54321 | Williams | Comp. Sci. | 54 | CS-190 | 2 | Spring | 2017 | B+ |
| 55739 | Sanchez | Music | 38 | MU-199 | 1 | Spring | 2018 | A- |
| 76543 | Brown | Comp. Sci. | 58 | CS-101 | 1 | Fall | 2017 | A |
| 76543 | Brown | Comp. Sci. | 58 | CS-319 | 2 | Spring | 2018 | A |
| 76653 | Aoi | Elec. Eng. | 60 | EE-181 | 1 | Spring | 2017 | C |
| 98765 | Bourikas | Elec. Eng. | 98 | CS-101 | 1 | Fall | 2017 | C- |
| 98765 | Bourikas | Elec. Eng. | 98 | CS-315 | 1 | Spring | 2018 | B |
| 98988 | Tanaka | Biology | 120 | BIO-101 | 1 | Summer | 2017 | A |
| 98988 | Tanaka | Biology | 120 | BIO-301 | 1 | Summer | 2018 | *null* |

©2026 - Tim Pengajar IF2040 Pemodelan Basis Data

# Dangerous in Natural Join

Beware of unrelated attributes with same name which get equated incorrectly

Example -- List the names of students along with the titles of courses that they have taken

- Correct version

```
select name, title
from student natural join takes, course
where takes.course_id = course.course_id;
```

- Incorrect version

```
select name, title
from student natural join takes natural join course;
```

  - This query omits all (student name, course title) pairs where the student takes a course in a department other than the student's own department.
  - The correct version (above), correctly outputs such pairs.



KNOWLEDGE & SOFTWARE ENGINEERING

# Outer Join

An extension of the join operation that avoids loss of information.

Computes the join and then adds tuples form one relation that does not match tuples in the other relation to the result of the join.

Uses *null* values.

Three forms of outer join:

- left outer join
- right outer join
- full outer join



KNOWLEDGE & SOFTWARE ENGINEERING

# Outer Join Examples

Relation *course*

| *course_id* | *title* | *dept_name* | *credits* |
|---|---|---|---|
| BIO-301 | Genetics | Biology | 4 |
| CS-190 | Game Design | Comp. Sci. | 4 |
| CS-315 | Robotics | Comp. Sci. | 3 |

Relation *prereq*

| *course_id* | *prereq_id* |
|---|---|
| BIO-301 | BIO-101 |
| CS-190 | CS-101 |
| CS-347 | CS-101 |

Observe that

*course* information is missing CS-347

*prereq* information is missing CS-315



KNOWLEDGE & SOFTWARE ENGINEERING

# Left Outer Join

*course* natural left outer join *prereq*

| *course_id* | *title* | *dept_name* | *credits* | *prereq_id* |
|---|---|---|---|---|
| BIO-301 | Genetics | Biology | 4 | BIO-101 |
| CS-190 | Game Design | Comp. Sci. | 4 | CS-101 |
| CS-315 | Robotics | Comp. Sci. | 3 | *null* |

In relational algebra: *course* ⟕ *prereq*



KNOWLEDGE & SOFTWARE ENGINEERING

# Right Outer Join

*course* natural right outer join *prereq*

| *course_id* | *title* | *dept_name* | *credits* | *prereq_id* |
|---|---|---|---|---|
| BIO-301 | Genetics | Biology | 4 | BIO-101 |
| CS-190 | Game Design | Comp. Sci. | 4 | CS-101 |
| CS-347 | *null* | *null* | *null* | CS-101 |

In relational algebra: *course* ⟖ *prereq*



KNOWLEDGE & SOFTWARE ENGINEERING

# Full Outer Join

*course* natural full outer join *prereq*

| *course_id* | *title* | *dept_name* | *credits* | *prereq_id* |
|---|---|---|---|---|
| BIO-301 | Genetics | Biology | 4 | BIO-101 |
| CS-190 | Game Design | Comp. Sci. | 4 | CS-101 |
| CS-315 | Robotics | Comp. Sci. | 3 | *null* |
| CS-347 | *null* | *null* | *null* | CS-101 |

In relational algebra: *course* ⟗ *prereq*



KNOWLEDGE & SOFTWARE ENGINEERING

# Joined Types and Conditions

**Join operations** take two relations and return as a result another relation.

These additional operations are typically used as subquery expressions in the **from** clause

**Join condition** – defines which tuples in the two relations match.

**Join type** – defines how tuples in each relation that do not match any tuple in the other relation (based on the join condition) are treated.

| *Join types* |
| --- |
| **inner join** |
| **left outer join** |
| **right outer join** |
| **full outer join** |

| *Join conditions* |
| --- |
| **natural** |
| **on** <predicate> |
| **using** $(A_1, A_2, \ldots, A_n)$ |



KNOWLEDGE & SOFTWARE ENGINEERING

# Joined Relations – Examples

*course* natural right outer join *prereq*

| *course_id* | *title* | *dept_name* | *credits* | *prereq_id* |
|---|---|---|---|---|
| BIO-301 | Genetics | Biology | 4 | BIO-101 |
| CS-190 | Game Design | Comp. Sci. | 4 | CS-101 |
| CS-347 | *null* | *null* | *null* | CS-101 |

*course* full outer join *prereq* using (*course_id*)

| *course_id* | *title* | *dept_name* | *credits* | *prereq_id* |
|---|---|---|---|---|
| BIO-301 | Genetics | Biology | 4 | BIO-101 |
| CS-190 | Game Design | Comp. Sci. | 4 | CS-101 |
| CS-315 | Robotics | Comp. Sci. | 3 | *null* |
| CS-347 | *null* | *null* | *null* | CS-101 |



KNOWLEDGE & SOFTWARE ENGINEERING

# Joined Relations – Examples

*course* inner join *prereq* on
*course.course_id = prereq.course_id*

| *course_id* | *title* | *dept_name* | *credits* | *prereq_id* | *course_id* |
|---|---|---|---|---|---|
| BIO-301 | Genetics | Biology | 4 | BIO-101 | BIO-301 |
| CS-190 | Game Design | Comp. Sci. | 4 | CS-101 | CS-190 |

What is the difference between the above, and a natural join?

*course* left outer join *prereq* on
*course.course_id = prereq.course_id*

| *course_id* | *title* | *dept_name* | *credits* | *prereq_id* | *course_id* |
|---|---|---|---|---|---|
| BIO-301 | Genetics | Biology | 4 | BIO-101 | BIO-301 |
| CS-190 | Game Design | Comp. Sci. | 4 | CS-101 | CS-190 |
| CS-315 | Robotics | Comp. Sci. | 3 | *null* | *null* |



KNOWLEDGE & SOFTWARE ENGINEERING

# Joined Relations – Examples

*course* natural right outer join *prereq*

| *course_id* | *title* | *dept_name* | *credits* | *prereq_id* |
|---|---|---|---|---|
| BIO-301 | Genetics | Biology | 4 | BIO-101 |
| CS-190 | Game Design | Comp. Sci. | 4 | CS-101 |
| CS-347 | *null* | *null* | *null* | CS-101 |

*course* full outer join *prereq* using (*course_id*)

| *course_id* | *title* | *dept_name* | *credits* | *prereq_id* |
|---|---|---|---|---|
| BIO-301 | Genetics | Biology | 4 | BIO-101 |
| CS-190 | Game Design | Comp. Sci. | 4 | CS-101 |
| CS-315 | Robotics | Comp. Sci. | 3 | *null* |
| CS-347 | *null* | *null* | *null* | CS-101 |



KNOWLEDGE & SOFTWARE ENGINEERING

# Views

# Views

In some cases, it is not desirable for all users to see the entire logical model (that is, all the actual relations stored in the database.)

Consider a person who needs to know an instructors name and department, but not the salary. This person should see a relation described, in SQL, by

```
select ID, name, dept_name
 from instructor
```

A **view** provides a mechanism to hide certain data from the view of certain users.

Any relation that is not of the conceptual model but is made visible to a user as a "virtual relation" is called a **view**.



KNOWLEDGE & SOFTWARE ENGINEERING

# View Definition

A view is defined using the **create view** statement which has the form

**create view** *v* **as** < query expression >

where <query expression> is any legal SQL expression. The view name is represented by *v*.

Once a view is defined, the view name can be used to refer to the virtual relation that the view generates.

View definition is not the same as creating a new relation by evaluating the query expression

- Rather, a view definition causes the saving of an expression; the expression is substituted into queries using the view.



KNOWLEDGE & SOFTWARE ENGINEERING

# View Definition and Use

A view of instructors without their salary

```
create view faculty as
    select ID, name, dept_name
     from instructor
```

Find all instructors in the Biology department

```
select name
from faculty
where dept_name = 'Biology'
```

Create a view of department salary totals

```
create view departments_total_salary(dept_name, total_salary) as
    select dept_name, sum (salary)
     from instructor
    group by dept_name;
```



KNOWLEDGE & SOFTWARE ENGINEERING

# Views Defined Using Other Views

One view may be used in the expression defining another view

A view relation $v_1$ is said to ***depend directly*** on a view relation $v_2$ if $v_2$ is used in the expression defining $v_1$

A view relation $v_1$ is said to ***depend on*** view relation $v_2$ if either $v_1$ depends directly to $v_2$ or there is a path of dependencies from $v_1$ to $v_2$

A view relation $v$ is said to be ***recursive*** if it depends on itself.



KNOWLEDGE & SOFTWARE ENGINEERING

# Views Defined Using Other Views

```
create view physics_fall_2017 as
   select course.course_id, sec_id, building, room_number
   from course, section
   where course.course_id = section.course_id
            and course.dept_name = 'Physics'
            and section.semester = 'Fall'
            and section.year = '2017';


create view physics_fall_2017_watson as
    select course_id, room_number
    from physics_fall_2017
    where building= 'Watson';
```



KNOWLEDGE & SOFTWARE ENGINEERING

# Materialized Views

Certain database systems allow view relations to be physically stored.

- Physical copy created when the view is defined.
- Such views are called **Materialized view**:

If relations used in the query are updated, the materialized view result becomes out of date

- Need to **maintain** the view, by updating the view whenever the underlying relations are updated.



KNOWLEDGE & SOFTWARE ENGINEERING

# Modification of the Database

# Modification of the Database

**Deletion** of tuples from a given relation.

**Insertion** of new tuples into a given relation

**Updating** of values in some tuples in a given relation



KNOWLEDGE & SOFTWARE ENGINEERING

# Deletion

Delete all instructors

```
delete from instructor
```

Delete all instructors from the Finance department

```
delete from instructor
where dept_name= 'Finance';
```

*Delete all tuples in the instructor relation for those instructors associated with a department located in the Watson building.*

```
delete from instructor
 where dept name in (select dept name
                     from department
                     where building = 'Watson');
```



KNOWLEDGE & SOFTWARE ENGINEERING

# Deletion (Cont.)

Delete all instructors whose salary is less than the average salary of instructors

```
delete from instructor
where salary < (select avg (salary)
                from instructor);
```

- Problem: as we delete tuples from *instructor*, the average salary changes
- Solution used in SQL:
  1. First, compute **avg** (salary) and find all tuples to delete
  2. Next, delete all tuples found above (without recomputing **avg** or retesting the tuples)



KNOWLEDGE & SOFTWARE ENGINEERING

# Insertion

Add a new tuple to *course*

```
insert into course
    values ('CS-437', 'Database Systems', 'Comp. Sci.', 4);
```

or equivalently

```
insert into course (course_id, title, dept_name, credits)
    values ('CS-437', 'Database Systems', 'Comp. Sci.', 4);
```

Add a new tuple to *student* with *tot_creds* set to null

```
insert into student
    values ('3003', 'Green', 'Finance', null);
```



KNOWLEDGE & SOFTWARE ENGINEERING

# Insertion (Cont.)

Make each student in the Music department who has earned more than 144 credit hours an instructor in the Music department with a salary of $18,000.

```
insert into instructor
        select ID, name, dept_name, 18000
        from   student
        where  dept_name = 'Music' and total_cred > 144;
```

the **select from where** statement is evaluated fully before any of its results are inserted into the relation.

Otherwise queries like

```
insert into table1 select * from table1
```

would cause problem



KNOWLEDGE & SOFTWARE ENGINEERING

# Updates

Give a 5% salary raise to all instructors

```
update instructor
    set salary = salary * 1.05
```

Give a 5% salary raise to those instructors who earn less than 70000

```
update instructor
    set salary = salary * 1.05
    where salary < 70000;
```

Give a 5% salary raise to instructors whose salary is less than average

```
update instructor
set salary = salary * 1.05
where salary < (select avg (salary)
                from instructor);
```



KNOWLEDGE & SOFTWARE ENGINEERING

# Updates (Cont.)

Increase salaries of instructors whose salary is over $100,000 by 3%, and all others by a 5%

- Write two **update** statements:

```
update instructor
    set salary = salary * 1.03
    where salary > 100000;
update instructor
    set salary = salary * 1.05
    where salary <= 100000;
```

- The order is important
- Can be done better using the **case** statement (next slide)



KNOWLEDGE & SOFTWARE ENGINEERING

# Case Statement for Conditional Updates

Same query as before but with case statement

```
update instructor
  set salary = case
                 when salary <= 100000 then salary * 1.05
                  else salary * 1.03
                 end
```



KNOWLEDGE & SOFTWARE ENGINEERING

# Updates with Scalar Subqueries

Recompute and update tot_creds value for all students

```
update student S
set tot_cred = (select sum(credits)
                from takes, course
                where takes.course_id = course.course_id  and
                      S.ID= takes.ID.and
                      takes.grade <> 'F' and
                       takes.grade is not null);
```

Sets *tot_creds* to null for students who have not taken any course

Instead of sum(*credits*), use:

```
case
 when sum(credits) is not null then sum(credits)
  else 0
end
```



KNOWLEDGE & SOFTWARE ENGINEERING

# Update of a View

Add a new tuple to *faculty* view which we defined earlier

**insert into** *faculty*

**values** ('30765', 'Green', 'Music');

This insertion must be represented by the insertion into the *instructor* relation

- Must have a value for salary.

Two approaches

- Reject the insert
- Insert the tuple

('30765', 'Green', 'Music', null)

into the *instructor* relation



KNOWLEDGE & SOFTWARE ENGINEERING

# Some Updates Cannot be Translated Uniquely

```
create view instructor_info as
        select ID, name, building
         from instructor, department
         where instructor.dept_name = department.dept_name;

insert into instructor_info

              values ('69987', 'White', 'Taylor');
```

Issues

- Which department, if multiple departments in Taylor?
- What if no department is in Taylor?



KNOWLEDGE & SOFTWARE ENGINEERING

# And Some Not at All

```
create view history_instructors as
   select *
   from instructor
   where dept_name= 'History';
```

What happens if we insert

('25566', 'Brown', 'Biology', 100000)

into *history_instructors?*



KNOWLEDGE & SOFTWARE ENGINEERING

# View Updates in SQL

Most SQL implementations allow updates only on simple views

- The **from** clause has only one database relation.
- The **select** clause contains only attribute names of the relation, and does not have any expressions, aggregates, or **distinct** specification.
- Any attribute not listed in the **select** clause can be set to null
- The query does not have a **group** by or **having** clause.

©2026 - Tim Pengajar IF2040 Pemodelan Basis Data

KNOWLEDGE & SOFTWARE ENGINEERING